MANUAL INTRODUCTORIO DE SVELTE

Framework para el desarrollo de interfaces web



3 DE OCTUBRE DE 2026

JORGE GARCÍA ORTIZ
2º DAW

Índice

1. ¿Qué es y para qué sirve? 2
1.1. Definición 2
1.2. ¿Para qué surgió y qué problemas resuelve? 2
1.3. ¿Quién la mantiene y cuál es su versión actual? 2
2. Conceptos clave 2
2.1. Componentes 2
2.2. Runes 2
2.3. Snippets 2
2.4. Props 3
3. Requisitos e instalación 3
3.1. Requisitos previos 3
3.2. Comandos de instalación 3
3.3. Estructura de carpetas generada 4
3.4. Funcionamiento final 5
4. Ejemplo básico de uso 6
4.1. Descripción del miniproyecto 6
4.2. Código explicado por partes 6
4.3. Resultado final 7
5. Ventajas, inconvenientes y conclusión 8
5.1. Ventajas 8
5.2. Inconvenientes 8
5.3. Conclusión 8
6. Bibliografía 8

1. ¿Qué es y para qué sirve?

1.1. Definición

Svelte es una tecnología que permite crear páginas y aplicaciones web interactivas. Utiliza HTML, CSS y JavaScript y se encarga de preparar el código para que funcione en el navegador.

1.2. ¿Para qué surgió y qué problemas resuelve?

Surgió para hacer más fácil la creación de páginas web interactivas, permitiendo trabajar de forma sencilla con HTML, CSS y JavaScript.

Svelte intenta hacer que el navegador tenga que trabajar menos, preparando parte del código antes de que se ejecute. Así, podemos crear aplicaciones eficientes sin tener que hacer tantas optimizaciones por nuestra cuenta.

1.3. ¿Quién la mantiene y cuál es su versión actual?

Svelte fue creado por Rich Harris. Actualmente, es un proyecto de código abierto mantenido por desarrolladores a tiempo completo y parcial, que cuenta con el apoyo de Vercel (empresa especializada en desarrollo y alojamiento web), siendo Svelte 5 la versión actual.

2. Conceptos clave

Antes de utilizar Svelte es necesario conocer algunos conceptos básicos que permiten entender cómo se crean y organizan las aplicaciones.

2.1. Componentes

Svelte permite crear archivos .svelte donde podemos escribir el HTML (lo que aparece en la página), JavaScript (lo que hace) y CSS (cómo se ve), todo junto.

Ejemplo: App.svelte → archivo principal de la aplicación.

2.2. Runes

Las runes son una característica de Svelte 5 que permite controlar los datos y cómo cambian en la página. Algunas de las más utilizadas son $state, $derived y $effect.

Por ejemplo, $state permite guardar un dato que puede cambiar. Cuando cambia, Svelte actualiza automáticamente el HTML que muestra ese dato, sin tener que modificarlo manualmente.

2.3. Snippets

Los snippets permiten crear un trozo de página con un nombre y reutilizarlo varias veces. Así no hay que escribir lo mismo muchas veces.

Se crea con #snippet y se usa con @render:

```
{#snippet saludo()}
      <p>Hola</p>
{/snippet}
{@render saludo()}
{@render saludo()}
```

2.4. Props

Las props permiten pasar información de un componente a otro. Así, un componente puede recibir datos y utilizarlos para mostrar diferentes contenidos.

Ejemplo: un componente recibe el nombre de un usuario y lo muestra en la página.

3. Requisitos e instalación

3.1. Requisitos previos

Para utilizar Svelte se necesita tener instalado:

- Node.js, para ejecutar las herramientas necesarias para crear y ejecutar el proyecto.
- npm, que se instala junto con Node.js y permite instalar las dependencias.
- Un editor de código, como Visual Studio Code.
- Un navegador web para comprobar el funcionamiento de la aplicación.

En mi caso he utilizado Node.js 24.21.0, npm 11.19.0 y Visual Studio Code.

3.2. Comandos de instalación

Para crear el proyecto se utiliza el siguiente comando:

```
jorge@DESKTOP-D38NIJ9 MINGW64 ~/Desktop/DAW/OPT/manual (main)
$ npx sv create svelte-manual
```

Nos hará una serie de preguntas para la instalación, yo he puesto las siguientes opciones:

```
◇ Which template would you like?
  SvelteKit minimal

◇ Add type checking with TypeScript?
  No

◇ What would you like to add to your project? (use arrow keys / space bar)
  none

◆ Project created

◇ Detected package managers. Which one should we use to install dependencies?
  npm
```

Después, accedemos a la carpeta del proyecto:

```
jorge@DESKTOP-D38NIJ9 MINGW64 ~/Desktop/DAW/OPT/manual (main)
$ cd svelte-manual
```

Y con el siguiente comando instalamos las dependencias necesarias:

```
jorge@DESKTOP-D38NIJ9 MINGW64 ~/Desktop/DAW/OPT/manual/svelte-manual (main)
$ npm install

> svelte-manual@0.0.1 prepare
> svelte-kit sync || echo ''


removed 1 package, and audited 48 packages in 2s

10 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Por último, se inicia el proyecto con:

```
jorge@DESKTOP-D38NIJ9 MINGW64 ~/Desktop/DAW/OPT/manual/svelte-manual (main)
$ npm run dev

> svelte-manual@0.0.1 dev
> vite dev

20:21:46 [vite] (ssr) connected.
20:21:46 [vite] (client) Forced re-optimization of dependencies

  VITE v8.3.2  ready in 1865 ms

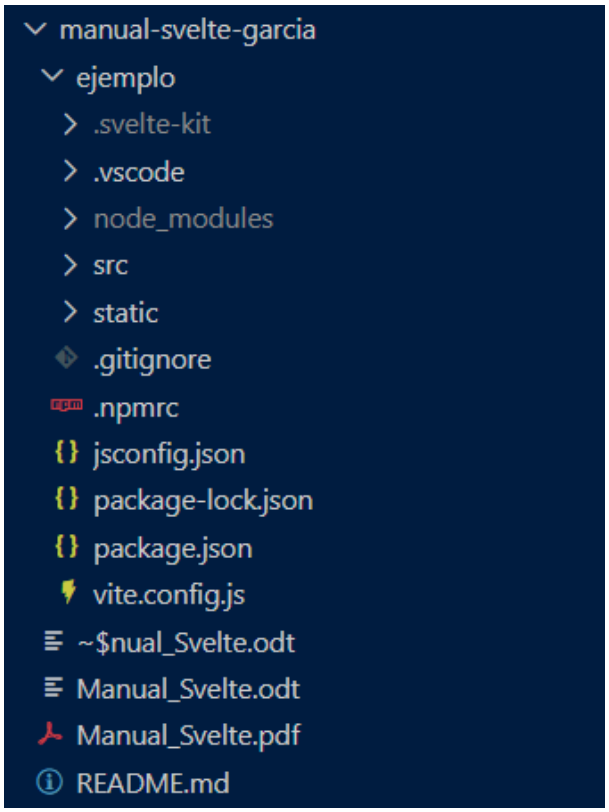
  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
```

3.3. Estructura de carpetas generada

Al crear el proyecto se genera una estructura de carpetas y archivos:

- src/: contiene el código de la aplicación.
- routes/: contiene las páginas.
- +page.svelte: página principal.
- static/: contiene archivos estáticos.
- package.json: contiene información del proyecto y sus dependencias.
- svelte.config.js: contiene la configuración de Svelte.

En mi caso la estructura de carpetas es de la siguiente forma:

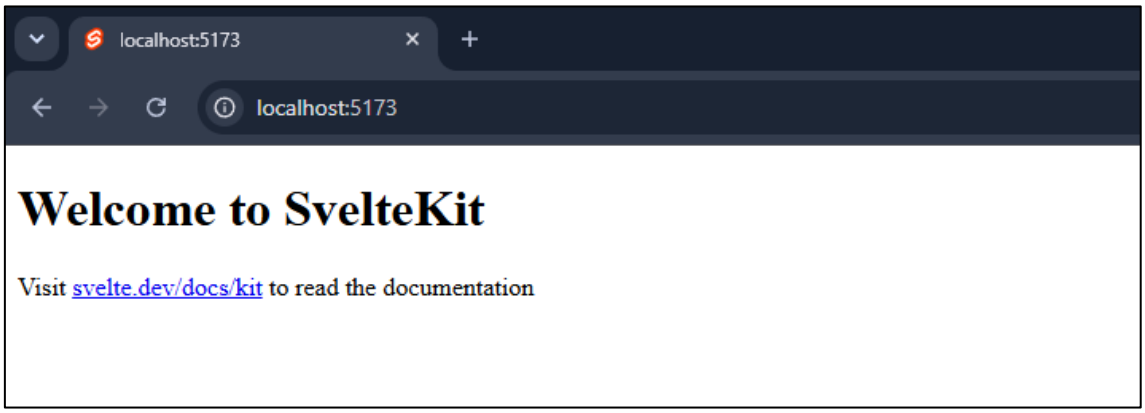


3.4. Funcionamiento final

Para comprobar que la instalación funciona correctamente, se inicia el proyecto con:

```
npm run dev
```

Y abrimos en el navegador la dirección proporcionada por la terminal:



4. Ejemplo básico de uso

4.1. Descripción del mini proyecto

He creado un pequeño proyecto en el que podemos añadir tareas a una lista. Cada tarea que añadimos se muestra automáticamente en la página.

4.2. Código explicado por partes

Todo el código se encuentra el fichero +page.svelte, donde lo tengo dividido en dos partes principales: una parte con el script, donde se encuentra la lógica del programa:

```
<script>
    // Creamos una variable para guardar el texto que escribimos en el input
    // $state() -> con esta runa le decimos a Svelte que este dato puede cambiar con el tiempo
    let nuevaTarea = $state("");

    // Creamos un array donde meteremos todas las tareas
    // $state() -> permite que Svelte detecte los cambios y actualice la página
    let arrayTareas = $state([]);

    // Creamos la función que agrega la tarea
    function agregarTarea() {

        // Solo añadimos la tarea si el texto no está vacío
        if (nuevaTarea !== "") {

            // Añadimos la tarea al array de tareas
            arrayTareas.push(nuevaTarea);

            // Vaciamos el input después de añadir la tarea
            nuevaTarea = "";
        }
    }
</script>
```

Y la otra donde se encuentra la estructura que se muestra en la página.

```
<!-- Escribimos el título de la aplicación -->
<h1>Lista de tareas</h1>

<!-- Ponemos un campo para escribir la tarea a realizar -->
<!-- bind:value={nuevaTarea} -> esto enlaza el valor del input con el de la variable 'nuevaTarea' -->
<input type="text" bind:value={nuevaTarea} placeholder="Escribe una tarea">

<!-- Ponemos un botón para añadir esa tarea -->
<!-- onclick={agregarTarea} -> al darle click al boton ejecutamos la función que crea la nueva tarea -->
<button onclick={agregarTarea}>Añadir</button>

<!-- Creamos una lista con todas las tareas que tenemos en el array -->
<ul>
    <!--  Recorremos todas las tareas del array -->
    <!--  Este es el equivalente a un bucle 'for of' en JavaScript -->
    {#each arrayTareas as tarea}

        <!-- Mostramos la tarea -->
        <!-- {} -> nos permite decirle a Svelte que lo que hay dentro es una variable con un valor -->
        <!-- tarea -> contiene el valor actual del array de tareas -->
        <li>{tarea}</li>

    {/each}
</ul>
```

4.3. Resultado final

Para comprobar el resultado, accederemos a la URL que nos proporciona la propia terminal al ejecutar el proyecto.

Por lo tanto, escribiremos algo en el input y pulsaremos en el botón añadir:



Y veremos en la parte de abajo como nuestra lista de tareas se va completando:



5. Ventajas, inconvenientes y conclusión

5.1. Ventajas

- Permite trabajar con HTML, CSS y JavaScript dentro de los componentes .svelte.
- La reactividad actualiza la página cuando cambian los datos.
- Permite reutilizar componentes y snippets.

5.2. Inconvenientes

- Es algo confuso y complicado de aprender.
- En proyectos grandes puede ser necesario aprender también SvelteKit.

5.3. Conclusión

Después de conocer Svelte, considero que es una gran tecnología para crear aplicaciones web interactivas. Me gusta que se pueda trabajar con HTML, CSS y JavaScript de forma directa.

Sí utilizaría Svelte en un proyecto real, aunque solo en algunas partes del proyecto, especialmente en aquellas que necesiten más interactividad.

6. Bibliografía

Para la realización de este manual se han utilizado las siguientes fuentes de información y recursos:

- Página oficial de Svelte: https://svelte.dev/
- Documentación oficial de Svelte: https://svelte.dev/docs
- Anuncio oficial de Svelte 5: https://svelte.dev/blog/svelte-5-is-alive
- Repositorio oficial de Svelte (GitHub): https://github.com/sveltejs/svelte
- Versiones oficiales de Svelte: https://github.com/sveltejs/svelte/releases
- Componentes: https://svelte.dev/docs/svelte/svelte-files
- Runes: https://svelte.dev/search?q=runes
- Snippets: https://svelte.dev/docs/svelte/snippet
- Props: https://svelte.dev/docs/svelte/$props
- Svelte CLI - creación e instalación de proyectos: https://svelte.dev/docs/cli/overview
- SvelteKit - creación de proyectos: https://svelte.dev/docs/kit/creating-a-project
- Documentación oficial de Svelte - {#each}: https://svelte.dev/docs/svelte/each
- Svelte - Playground: https://svelte.dev/playground/hello-world?version=5.57.1
- Svelte - Tutorial interactivo:

  https://svelte.dev/tutorial/svelte/welcome-to-svelte
- Bluuweb – Fundamentos de Svelte:

  https://bluuweb.github.io/svelte-youtube/01-bases/
- Silvestre Vivo: https://youtube.com/playlist?list=PLtEoLIi1AQhWzcCE76KQi4OLHqlDu_XUh&si=LMOLVQF-0eyfcwll

El código completo del proyecto está disponible en mi repositorio de GitHub